Flow matching and Diffusion Models

Lecture 4: Building an Image Generator

Guidance for Flow Models

Unconditional (Unguided): "Generate an image"

Conditional (Guided): "Generate an image of a cat baking a cake."

Unguided	Guided
Unguided marginal probability path: $p_t(x)$	Guided marginal probability path: $p_t(x\|y)$
Unguided marginal vector field: $u_t^{target}(x)$	Guided marginal vector field: $u_t^{target}(x\|y)$
Unguided marginal: $\nabla \log p_t(x)$	Guided marginal: $\nabla \log p_t(x\|y)$
Unguided model: $u_t^\theta(x)$	Guided model: $u_t^\theta(x\|y)$
Unguided CFM Objective $\mathcal{L}_{CFM}(\theta)$	Guided CFM Objective ??

Guided conditional flow matching object

if 1 fixed y, we recover unguided problem:

$$\mathcal{L}_{CFM}^{guided}(\theta; y) = \mathbb{E}_{z\sim p_{data}(z|y),\, t\sim Unif[0,1),\, x\sim p_t(\cdot|z)} \| u_t^\theta(x|y) - u_t^{target}(x|z) \|^2$$

let y vary:

$$\mathcal{L}_{CFM}^{guided}(\theta) = \mathbb{E}_{(z,y)\sim p_{data}(z,y),\, t\sim Unif[0,1),\, x\sim p_t(\cdot|z)} \| u_t^\theta(x|y) - u_t^{target}(x|z) \|^2$$

Algorithm 7 Guided Sampling Procedure

Require: A trained guided vector field $u_t^\theta(x|y)$.
1: Select a prompt $y \in \mathcal{Y}$, such as "a cat baking a cake".
2: Initialize $X_0 \sim p_{\text{init}}$.
3: Simulate $\mathrm{d}X_t = u_t^\theta(X_t|y)\mathrm{d}t$ from $t = 0$ to $t = 1$.

Classifier-free guidance

Gaussian cond. prob. path. $p_t(x|z) \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$

$$u_t^{target}(x|y) = a_t x + b_t \nabla \log p_t(x|y) \qquad (a_t, b_t) = \left(\frac{\dot\alpha_t}{\alpha_t},\ \frac{\dot\alpha_t \beta_t^2 - \dot\beta_t \beta_t \alpha_t}{\alpha_t} \right)$$ proven in Lecture 3

Bayes' rule $\Rightarrow$ $\nabla \log p_t(x|y) = \nabla \log \frac{p_t(x)p_t(y|x)}{p_t(y)} = \nabla \left(\log p_t(x) + \log p_t(y|x) - \log p_t(y) \right)$

∇_x

$$= \nabla \log p_t(x) + \nabla \log p_t(y|x)$$

$$\Rightarrow u_t^{target}(x|y) = a_t x + b_t [\nabla \log p_t(x) + \nabla \log p_t(y|x)]$$

$$= [a_t x + b_t \nabla \log p_t(x)] + b_t \nabla \log p_t(y|x) \qquad = u_t^{target}(x)$$

$$= u_t^{target}(x) + b_t \nabla \log p_t(y|x) \rightarrow \text{classifier}$$

choose "guidance scale" $w > 1$

$$\tilde u_t(x|y) = u_t^{target}(x) + b_t w \nabla \log p_t(y|x) = u_t^{target}(x) + b_t w \left(\nabla \log p_t(x|y) - \nabla \log p_t(x) \right)$$

$$= u_t^{target}(x) - w a_t x + w a_t x + b_t w \left(\nabla \log p_t(x|y) - \nabla \log p_t(x) \right)$$

$$= u_t^{target}(x) - w \left(a_t x + b_t \nabla \log p_t(x) \right) + w \left(a_t x + b_t \nabla \log p_t(x|y) \right)$$

$u_t^{target}(x)$ $\qquad$ $u_t^{target}(x|y)$

$$= (1-w) u_t^{target}(x) + w u_t^{target}(x|y)$$

Classifier-free Guidance Training

We may treat the unguided vector field as conditioned on nothing, but nothing is something.

$$u_t^{target}(x) = u_t^{target}(x|y=\varnothing)$$

we may now train a single model $u_t^\theta(x|y)$, $y \in \{y, \varnothing\}$ by re-using $\mathcal{L}_{CFM}^{guided}(\theta)$ and occasionally setting $y = \varnothing$

classifier-free guidance $\mathcal{L}_{CFM}^{CFG}(\theta) = \mathbb{E}_\square \| u_t^\theta(x|y) - u_t^{target}(x|z) \|^2$

$\square = (z, y) \sim p_{data}(z, y)$, with probability η: $y \leftarrow \varnothing$, $t \sim Unif[0,1)$, $x \sim p_t(\cdot|z)$

Algorithm 8 Classifier-Free Guidance Sampling Procedure

Require: A trained guided vector field $u_t^\theta(x|y)$.
1: Select a prompt $y \in \mathcal{Y}$, or take $y = \varnothing$ for unguided sampling.
2: Select a **guidance scale** $w > 1$.
3: Initialize $X_0 \sim p_{\text{init}}$.
4: Simulate $\mathrm{d}X_t = \left[(1-w)u_t^\theta(X_t|\varnothing) + w u_t^\theta(X_t|y)\right] \mathrm{d}t$ from $t = 0$ to $t = 1$.

Example: Gaussian probability path.

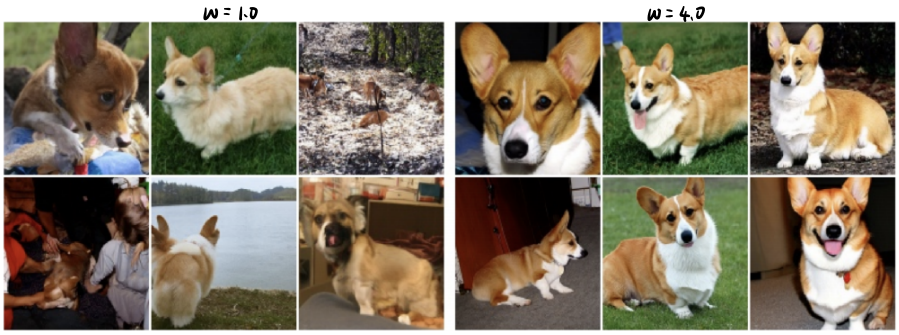
Algorithm 5 Classifier-free guidance training for Gaussian probability path $p_t(x|z) = \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d)$

Require: Paired dataset $(z, y) \sim p_{\text{data}}$, neural network u_t^θ

1: **for** each mini-batch of data **do**
2: Sample a data example (z, y) from the dataset.
3: Sample a random time $t \sim \text{Unif}_{[0,1]}$.
4: Sample noise $\epsilon \sim \mathcal{N}(0, I_d)$
5: Set $x = \alpha_t z + \beta_t \epsilon$
6: With probability p drop label: $y \leftarrow \varnothing$
7: Compute loss

$$\mathcal{L}(\theta) = \|u_t^\theta(x|y) - (\dot{\alpha}_t \epsilon + \dot{\beta}_t z)\|^2$$

8: Update the model parameters θ via gradient descent on $\mathcal{L}(\theta)$.
9: **end for**



So: when w is close to 1, the model relies primarily on the unconditional distribution learned during training — that is, what a realistic image looks like in the absence of external guidance. Conversely, the higher the w, the more the generated image aligns with the text prompt.

Guidance for Diffusion Models

$$\mathcal{L}_{\text{CSM}}^{\text{guided}}(\theta) = \mathbb{E}_{\square}\left[\|s_t^\theta(x|y) - \nabla \log p_t(x|z)\|^2\right] \qquad \square = (z,y) \sim p_{\text{data}}(z,y),\ t \sim \text{Unif},\ x \sim p_t(\cdot|z)$$

$$X_0 \sim p_{\text{init}},\quad dX_t = \left[u_t^\theta(X_t|y) + \frac{\sigma_t^2}{2} s_t^\theta(X_t|y)\right]dt + \sigma_t dW_t$$

Classifier-free guidance:

$$\nabla \log p_t(x|y) = \nabla \log p_t(x) + \nabla \log p_t(y|x) \qquad (s_t(x|y))$$

for guidance scale $w > 1$:

$$\begin{aligned}\tilde{s}_t(x|y) &= \nabla \log p_t(x) + w\,\nabla \log p_t(y|x) \\ &= \nabla \log p_t(x) + w\,(\nabla \log p_t(x|y) - \nabla \log p_t(x)) \\ &= (1-w)\,\nabla \log p_t(x) + w\,\nabla \log p_t(x|y) \\ &= (1-w)\,\nabla \log p_t(x|\varnothing) + w\,\nabla \log p_t(x|y)\end{aligned}$$

guided conditional score matching objective:

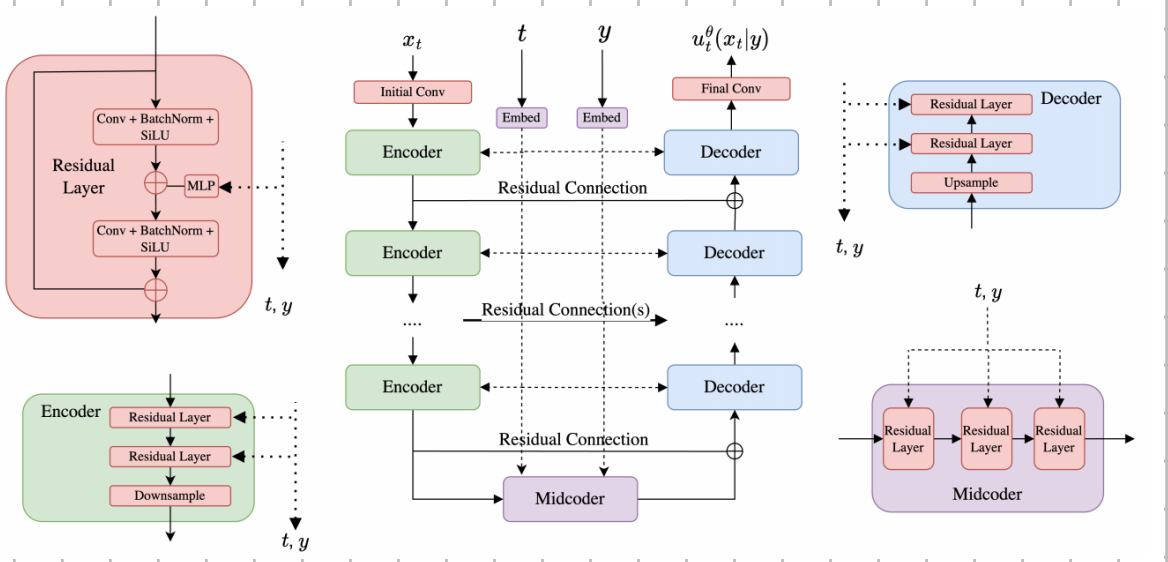
$$\mathcal{L}_{\text{DSM}}^{\text{CFG}}(\theta) = \mathbb{E}_{\square}\left[\|s_t^\theta(x|y) - \nabla \log p_t(x|z)\|^2\right] \qquad \square = (z,y) \sim p_{\text{data}}(z,y),\ t \sim \text{Unif},\ x \sim p_t(\cdot|z),\ \text{replace } y = \varnothing \text{ with prob. } \eta$$

U-Nets and Diffusion Transformers

U-Nets: a specific type of CNN (convolutional neural network)

- crucial feature: both its input and its output have the shape of images (probably with different number of channels)
  - eg. input: 256×256 output: 256×256

Diffusion models need to predict vector field $u_t^\theta(x|y)$ that has the same shape as the input x.



$x_t^{input} \in R^{3\times256\times256}$ Input to the U-Net

$x_t^{latent} = \mathcal{E}(x_t^{input}) \in R^{512\times32\times32}$ Pass through encoders to obtain latent

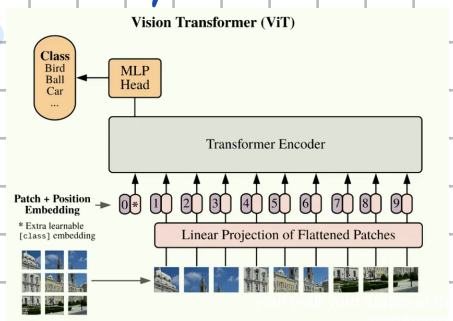
$x_t^{latent} = M(x_t^{latent}) \in R^{512\times32\times32}$ Pass latent through midcoder.

$x_t^{output} = \mathcal{D}(x_t^{latent}) \in R^{3\times256\times256}$ Pass through decoders to obtain output.

Diffusion Transformers (DiTs): dispense with convolutions and purely use attention.

base on Vision transformers (ViTs)

- divide an imag into patches
- embed each of these patches
- attend between the patches

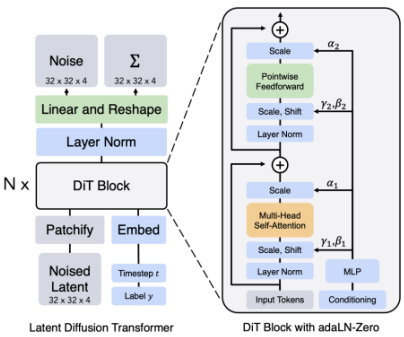


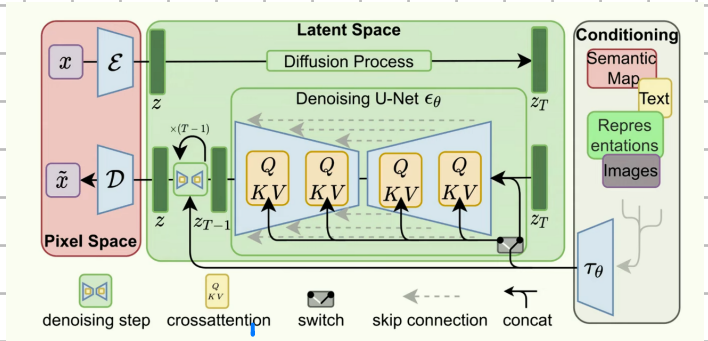
Latent diffusion models

To reduce memory usage, a common design pattern is to work in a latent space that can be considered a compressed version of our data at lower resolution.

Specifically, the usual approach is to combine a flow or diffusion model with a (variational) autoencoder

- encodes the training dataset in the latent space via an autoencoder
- training the flow or diffusion model in the latent space
- first sampling in the latent space using the trained flow or diffusion model, and then decoding of the output via the decoder.



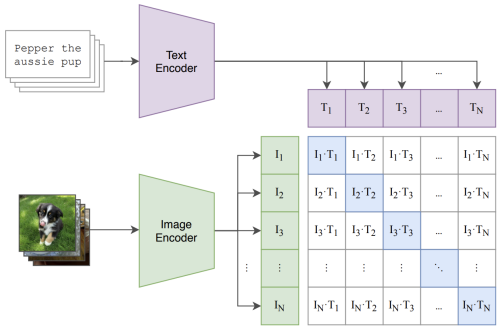


Encoding the Guiding Variable

- embedding raw input: ① y_{raw} is a discrete class-label.

$y_{raw} \in \mathcal{Y} \triangleq \{0, \ldots, N\}$: it's often easiest to simply learn a separate embedding vector for each of the N+1 possible values of y_{raw}, and set y to this embedding vector.

② y_{raw} is a text-prompt: $y = CLIP(y_{raw}) \in R^{d_{CLIP}}$



- Feeding in the embedding.

$y \in \mathbb{R}^d$ $x_t^{intermediate} \in \mathbb{R}^{C \times H \times W}$ the image information currently being processed by the model.

$y = MLP(y) \in \mathbb{R}^C$ map y from $\mathbb{R}^{dy}$ to $\mathbb{R}^c$

$y = reshape(y) \in \mathbb{R}^{C \times 1 \times 1}$ reshape y to "look" like an image.

$x_t^{intermediate} = broadcast\_add(x_t^{intermediate}, y) \in \mathbb{R}^{C \times H \times W}$ add y to $x_t^{intermediate}$ pointwise.

One exception to this simple pointwise conditioning scheme is when we have a sequence of embeddings as produced by some pretrained language model. In this case, we might consider using some sort of cross-attention scheme between our image (suitably patchified) and the tokens of the embedded sequence.

Stable Diffusion 3:

- SD3 makes use of different types of text embedding:
  - CLIP: provide a coarse, overarching embedding of the input text.
  - T6-XXL: provide a more granular level of context, allowing the model attends to particular elements of the conditioning text.
- MM-DiT (multi-modal DiT)